

Fundamentos de la Inteligencia Artificial (IA) en Ciberseguridad

Bienvenidos a esta sesión de introducción a los conceptos clave de la **Inteligencia Artificial (IA)**, un área fundamental cuya comprensión es imprescindible en el contexto de un máster en ciberseguridad. A lo largo de este módulo, profundizaremos en la materia, pero esta sesión servirá como una base sólida y estructurada.

Conceptos Clave de la Inteligencia Artificial (IA)

La Inteligencia Artificial se define como la capacidad de los sistemas informáticos para **emular procesos cognitivos humanos**, incluyendo el razonamiento, el aprendizaje, la percepción y la toma de decisiones. En esencia, la IA permite a las máquinas ejecutar tareas que habitualmente requerirían inteligencia humana, como la comprensión del lenguaje natural o el reconocimiento de patrones complejos.

Rol y Aplicación de la IA

La tecnología de la IA no es un concepto futurista, sino que ya se encuentra **integrada en nuestra vida diaria y en entornos profesionales**, transformando profundamente el mundo digital.

- **Emulación Cognitiva:** La IA puede imitar procesos cognitivos humanos, **aprendiendo** y tomando decisiones basadas en la **percepción del entorno**.
- **Transformación Digital:** **Cambia la forma en que interactuamos** con dispositivos y servicios digitales, haciendo estas interacciones más inteligentes, personalizadas y eficientes.
- **Eficiencia y Calidad de Vida:** Al mejorar la eficiencia de los procesos profesionales, la IA contribuye a optimizar la calidad de vida en diversos ámbitos.

- **Análisis de Datos Complejos:** Es capaz de **identificar patrones en grandes conjuntos de datos** y tomar decisiones informadas a partir de estos análisis. Esto resulta crucial para abordar y solucionar problemas en situaciones que son **demasiado complejas** para los enfoques tradicionales de computación.
-

Clasificación de los Tipos de IA

De forma general, la IA se clasifica en tres categorías principales, dependiendo de su capacidad y alcance:

1. Narrow AI (IA Estrecha o Débil):

- **Propósito:** Es la IA que existe actualmente y está diseñada para **tareas específicas y concretas**.
- **Especialización:** Se especializa solo en un aspecto o una función muy concreta.
- **Ejemplo Práctico:** Sistemas de **reconocimiento de voz** (como asistentes virtuales), filtros de *spam* o sistemas de recomendación de contenido.

2. General AI (IA General o Fuerte):

- **Estado:** Actualmente es un concepto **teórico**, aunque el progreso tecnológico nos acerca cada vez más a su realización.
- **Capacidad:** Sería capaz de **desempeñar cualquier tarea intelectual** que un ser humano podría hacer, incluyendo la abstracción, el razonamiento y la solución de problemas generales.

3. Superintelligence (Superinteligencia):

- **Definición:** Es una forma avanzada de IA que **supera la capacidad cognitiva humana** en todos los aspectos, incluyendo la creatividad y la habilidad de resolución de problemas a un nivel que va más allá de la comprensión humana actual.
-

Machine Learning (Aprendizaje Automático)

El **Machine Learning (ML)** o **Aprendizaje Automático** es una subdisciplina de la Inteligencia Artificial que ha ganado un gran protagonismo.

- **Principio Fundamental:** Permite que las máquinas **aprendan de los datos sin una programación explícita** (*"sin que se les instruya específicamente cómo hacerlo"*). Es decir, utiliza algoritmos para que las máquinas mejoren su rendimiento con la experiencia.
 - **Metodología:** Emplea **algoritmos matemáticos** para identificar patrones en los datos.
 - **Beneficio Clave:** A través del análisis de estos datos, el aprendizaje automático permite realizar **predicciones precisas** y tomar **decisiones informadas** y relevantes, transformando la toma de decisiones en múltiples sectores.
-

Redes Neuronales y Deep Learning

Un elemento clave en todas las arquitecturas modernas de Inteligencia Artificial son las **Redes Neuronales**.

Definición y Propósito

- **Modelo Computacional:** Son **modelos computacionales que imitan la estructura y el funcionamiento del cerebro humano**. Están diseñados para replicar la forma en que el cerebro procesa la información y detecta relaciones complejas.
- **Base del Deep Learning:** Son fundamentales en el campo del **Aprendizaje Profundo** (*Deep Learning*), un subconjunto avanzado del Machine Learning.
- **Capacidad de Reconocimiento:** El *Deep Learning*, asistido por las redes neuronales, mejora significativamente la capacidad de las máquinas para **reconocer patrones complejos** que serían invisibles para otros métodos.

Aplicaciones Críticas

Las redes neuronales juegan un papel crucial en áreas de alta criticidad, como:

- **Detección de Fraude:** Identificando actividades financieras sospechosas o fraudulentas.
 - **Diagnóstico Médico:** Analizando grandes conjuntos de datos médicos (imágenes, historiales) para asistir en diagnósticos más precisos.
 - **Ciberseguridad:** Reconociendo patrones de tráfico anómalos o *malware* en tiempo real.
-

Conclusiones y Proyecciones

La Inteligencia Artificial representa una **redefinición de la interacción humana con la tecnología**, imitando procesos cognitivos y ofreciendo soluciones inteligentes que abarcan desde la vida diaria hasta el ámbito profesional.

- **Evolución:** La IA avanza desde la **IA estrecha** con habilidades muy específicas, hacia la visión futurista de una **superinteligencia**, ampliando constantemente su impacto.
- **Motores de la Innovación:** El **aprendizaje automático** y las **redes neuronales** son los motores de esta innovación, analizando datos y patrones para impulsar avances en campos como la **salud** y, de manera crucial para este máster, la **ciberseguridad**.
- **Panorama Futuro:** Esta evolución promete **eficiencia y precisión sin precedentes** en la resolución de problemas. No obstante, como toda tecnología transformadora, también **plantea desafíos éticos y de implementación** que deben ser abordados de manera proactiva, marcando el inicio de una nueva y compleja era en el desarrollo tecnológico.

Herramientas Fundamentales para el Desarrollo de Proyectos de IA en Ciberseguridad

Bienvenidos a esta nueva sesión, donde abordaremos el conjunto de **herramientas básicas** que conformarán la infraestructura de trabajo para todos los ejercicios prácticos en este máster. Aunque existen muchas más, estas librerías y aplicaciones constituyen la base troncal para el desarrollo de la Inteligencia Artificial (IA) aplicada a la ciberseguridad.

Descripción del Conjunto de Herramientas Esenciales

El ecosistema de la IA en Python se sustenta en un conjunto de bibliotecas poderosas, cada una con un rol específico:

1. **Conda (o Miniconda):** Es el gestor de paquetes y de entornos fundamental. Facilita la **instalación de software y sus dependencias** de forma aislada, previniendo conflictos de compatibilidad entre diferentes proyectos.
2. **Jupyter Notebook:** Una **aplicación web** que proporciona un entorno interactivo crucial. Permite crear y compartir documentos que combinan **código en vivo**, ecuaciones, **visualizaciones** y texto narrativo (*markdown*), optimizando el análisis y la documentación.
3. **Pandas:** Una biblioteca de Python que proporciona estructuras de datos eficientes, siendo la más importante el **DataFrame**, y **herramientas de alto nivel para el análisis de datos** y su manipulación.
4. **NumPy:** Fundamental para la **computación científica en Python**. Proporciona soporte para la gestión de **grandes arrays y matrices multidimensionales**, junto con una colección de funciones matemáticas optimizadas para operar con ellos.
5. **Scikit-Learn (sklearn):** La biblioteca de **aprendizaje automático (*Machine Learning*)** más popular en Python. Ofrece herramientas sencillas y eficientes

para el **análisis de datos y el modelado estadístico**, incluyendo algoritmos de clasificación, regresión y *clustering*.

6. **Matplotlib**: Una librería esencial para la **visualización de datos**. Permite crear todo tipo de gráficos estáticos, animados e interactivos, lo cual es primordial para entender la información y los resultados de los modelos de IA.

Procedimientos Prácticos: Instalación y Funcionamiento Básico

Para comenzar a familiarizarnos y sentar la base para ejercicios más avanzados, realizaremos la instalación de estas herramientas de forma secuencial, utilizando una máquina virtual (VM) de Linux Ubuntu limpia. Es crucial que sigamos este procedimiento, ya que esta VM será nuestro entorno de trabajo principal.

1. Instalación y Configuración de Conda / Miniconda (Gestor de Entornos)

Conda es fundamental en la ciencia de datos y el mundo de la IA, ya que permite **aislar proyectos** con diferentes dependencias y versiones de librerías, evitando problemas de compatibilidad que son habituales en la programación.

A. Instalación de Miniconda

1. **Descarga**: Abrir el navegador en la VM y buscar "Conda installer Linux" o "Miniconda Installer para Linux".
2. **Selección de Versión**: En la web oficial, se debe elegir la versión adecuada para la arquitectura del sistema. Por ejemplo, si se usa un Mac con chip M1 (arquitectura ARM), se debe seleccionar la versión **aarch64**. El archivo se descargará (ej: **Miniconda3-latest-Linux-aarch64.sh**) en la carpeta **Downloads**.

3. **Ejecución en Terminal:** Abrir una terminal y navegar hasta la carpeta donde se descargó el instalador:

```
cd downloads
```

```
bash Miniconda3-latest-Linux-aarch64.sh
```

4. **Aceptación de Licencia:** El instalador solicitará aceptar la licencia. Se debe escribir **yes** tras revisar los términos.
5. **Directorio de Instalación:** Se solicitará confirmar la ubicación de instalación (por defecto: `/home/user/miniconda3`). Se recomienda dejar la ubicación por defecto y confirmarla.
6. **Inicialización:** Dejar la opción por defecto para inicializar Conda, completando la instalación.

B. Creación y Activación del Entorno Aislado

El siguiente paso es crear un entorno específico para nuestros ejercicios de IA, denominado **ai**.

Creación del Entorno: Utilizamos el siguiente comando, donde el argumento `--name` especifica el nombre del nuevo entorno.

```
conda create --name ai
```

Listar Entornos (Opcional): Para verificar que el entorno haya sido creado, se puede usar:

```
conda env list
```

Activación del Entorno: Este comando es esencial, ya que nos permite **acceder y operar dentro de ese entorno cerrado**. Notarás que el *prompt* de la terminal cambia, mostrando **(ai)** a la izquierda, indicando que todas las instalaciones se realizarán dentro de este espacio controlado.

```
conda activate ai
```

Desactivación (Para salir): Para salir del entorno `ai` y volver al entorno base de la máquina, se usa:

```
conda deactivate
```

2. Instalación y Funcionamiento Básico de Jupyter Notebook

Jupyter Notebook es una herramienta crucial para el análisis de datos, ya que permite ejecutar el código de forma segmentada (en celdas) y visualizar los resultados de manera interactiva.

A. Instalación de Jupyter Notebook

Ya dentro del entorno (`ai`), la instalación se simplifica enormemente usando el gestor de paquetes de Conda.

Comando de Instalación:

```
conda install -c conda-forge notebook
```

- **Sintaxis Detallada:** `conda install` es el comando de instalación.
`-c conda-forge` especifica el canal *conda-forge*, una fuente comunitaria de paquetes confiables. `notebook` es el nombre del paquete.
- **Confirmación:** El instalador listará las dependencias y pedirá confirmación (escribir `yes` o `y`).

B. Preparación y Levantamiento del Servidor

1. **Creación del Directorio de Trabajo:** Es una buena práctica crear una carpeta dedicada para los ficheros del proyecto.

```
mkdir ai
```

```
cd ai
```

2. **Levantamiento del Notebook:** Una vez en la carpeta, se inicia el servidor web de Jupyter Notebook.

```
jupyter notebook
```

Esto abrirá automáticamente una pestaña en el navegador web (generalmente en `localhost`), mostrando el contenido de la carpeta actual (que estará vacía en este punto).

C. Interfaz Básica de Jupyter Notebook

Desde el navegador, se puede crear un nuevo *notebook* haciendo clic en **New** y seleccionando **Python 3** (o el *kernel* deseado).

- **Entorno Interactivo:** Jupyter funciona con **celdas**. Puedes escribir código Python en una celda y ejecutarla de forma independiente.
- **Botones Principales:** Los botones principales permiten ejecutar una celda, copiarla, insertarla, etc.
- **Versatilidad:** Es un entorno **versátil** ideal para la educación y el análisis de datos, permitiendo un flujo de trabajo flexible donde se puede modificar, re-ejecutar o inspeccionar el código por partes sin reiniciar toda la secuencia.

3. Instalación y Funcionamiento Básico de Pandas

Pandas es la navaja suiza para la manipulación de datos. Demostraremos su instalación y uso directamente desde el *notebook*.

A. Instalación de Pandas desde Jupyter

Jupyter Notebook permite ejecutar comandos del sistema (CLI) o de Conda/Pip directamente en una celda, usando el **signo de admiración (!)** al inicio.

Instalación Rápida (Recomendada): Para evitar la pregunta de confirmación (*prompting*), se añade el parámetro **-y**.

```
!conda install -y pandas
```

- **Propósito del !:** Indica al *kernel* de Jupyter que ejecute el comando como si fuera en la terminal (*shell*).
- **Propósito del -y:** Acepta automáticamente la instalación sin requerir la intervención del usuario.

B. Carga e Inspección de Datos con Pandas

El núcleo de Pandas es el **DataFrame**, una estructura de datos bidimensional con etiquetas (similar a una hoja de cálculo o tabla de base de datos), que permite la manipulación organizada. Para que el sistema reconozca los comandos de Pandas, es necesario importarlo primero.

Importación con Alias: Por convención, se importa con el alias **pd** (este debe ir en una celda **antes** de su uso).

```
import pandas as pd
```

- **Corrección de Errores en Jupyter:** Una de las ventajas de Jupyter es que si olvidas el *import* y obtienes un error (**NameError**), puedes simplemente añadir la celda de *import* encima de la que falló y re-ejecutar.

Lectura de un Fichero CSV: Suponiendo que el fichero **log2.csv** está en el directorio, Pandas lo lee y lo muestra automáticamente.

```
pd.read_csv('log2.csv')
```

- La ejecución de este comando carga el *dataset* en un **DataFrame**. Si el conjunto de datos es muy grande (ej. 65,531 entradas), Jupyter lo trunca para

mostrar solo las primeras filas, ofreciendo una vista rápida de la estructura del *dataset*.

4. Instalación y Funcionamiento Básico de NumPy

NumPy es esencial para la manipulación eficiente de datos numéricos y es la base de la mayoría de las librerías de IA.

A. Instalación e Importación

Aunque la instalación de Pandas a menudo incluye NumPy como dependencia, se debe asegurar su importación. Por convención, se importa con el alias `np`.

```
import numpy as np
```

B. Creación y Operación con Arrays

NumPy destaca por su objeto `ndarray` (array N-dimensional).

Creación de un Array: Se crea un array simple, que es la estructura de datos principal de NumPy.

```
a = np.array([1, 2, 3])
```

Operaciones Vectorizadas: Una de las grandes ventajas de NumPy es que permite aplicar operaciones matemáticas a todos los elementos del *array* de forma rápida y concisa (operaciones vectorizadas).

```
print("Array:", a)
```

```
print("Square:", a**2)
```

Resultado: Square: [1 4 9]

- La operación `a**2` calcula el cuadrado de cada elemento ($1^2=1$, $2^2=4$, $3^2=9$) de manera eficiente.

5. Instalación y Funcionamiento Básico de Scikit-Learn (sklearn)

Scikit-learn es la principal herramienta para el *Machine Learning* en Python.

A. Instalación

Para instalar Scikit-learn se utiliza el comando `pip install` precedido del signo de admiración, ya que es una práctica común en el desarrollo de IA y puede ser preferido para algunas librerías.

```
!pip install scikit-learn
```

B. Flujo de Trabajo Demostrativo con Random Forest (Clasificación)

A continuación, se presenta un flujo de trabajo rápido, segmentado en bloques, para demostrar cómo se usa Scikit-learn para crear, entrenar y evaluar un modelo de clasificación, aprovechando la ejecución parcial de Jupyter Notebook:

Preparación del Entorno (Importación): Se cargan los módulos necesarios para la tarea:

```
from sklearn import datasets
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.ensemble import RandomForestClassifier
```

```
from sklearn.metrics import accuracy_score
```

- **Propósito:** `datasets` carga datos de ejemplo (como el famoso Iris), `train_test_split` divide los datos, `RandomForestClassifier` es el algoritmo de clasificación, y `accuracy_score` es la métrica de evaluación.

Carga del Conjunto de Datos: Se carga el clásico conjunto de datos de las flores Iris.

```
iris = datasets.load_iris()
```

```
X, y = iris.data, iris.target
```

- **X:** Contiene las **Características** (mediciones de las flores).
- **y:** Contiene la **Variable Objetivo** (etiquetas o tipos de flores).

División del Conjunto de Datos: Se dividen los datos en conjuntos de **entrenamiento** y **prueba**.

```
X_train, X_test, y_train, y_test = train_test_split(X, y,  
test_size=0.3)
```

- **`train_test_split(...)`:** Asigna el 70% de los datos a entrenamiento (`X_train, y_train`) y el **30% a prueba** (`X_test, y_test`), una práctica crucial para una evaluación justa.

Creación y Entrenamiento del Modelo: Se instancia el algoritmo **Random Forest** y se entrena con los datos.

```
clf = RandomForestClassifier()
```

```
clf.fit(X_train, y_train)
```

- **clf.fit(...):** Este comando es el paso de **entrenamiento**, donde el modelo aprende los patrones a partir de los datos de entrenamiento.

Predicción y Evaluación (Precisión): Se utiliza el modelo entrenado para predecir los resultados de los datos de prueba (**X_test**) que nunca ha visto, y luego se compara la predicción (**y_pred**) con las etiquetas reales (**y_test**).

```
y_pred = clf.predict(X_test)
```

```
print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
# Output: Accuracy: 0.9555555555555556
```

- **Resultado:** El modelo predijo correctamente el tipo de flor en aproximadamente el **95.5%** de los casos, demostrando un alto rendimiento.

El valor de Jupyter Notebook aquí es evidente: permite al usuario **ejecutar y verificar el código por bloques**, facilitando la inspección del flujo de trabajo de *machine learning* y la depuración.

6. Instalación y Funcionamiento Básico de Matplotlib

Matplotlib es la librería que se encarga de la **visualización de la información**, una parte primordial en el mundo de la IA, ya que es la mejor forma de entender los datos, los modelos y sus resultados.

A. Importación y Visualización de un Gráfico de Dispersión

Para la visualización de datos, se utiliza el módulo `pyplot`, que por convención se importa como `plt`. Se utilizará el mismo *dataset* Iris para generar un gráfico de dispersión simple, que es fundamental en el análisis exploratorio de datos.

```
import matplotlib.pyplot as plt
from sklearn import datasets

# Carga del dataset y selección de dos características
(features)
iris = datasets.load_iris()
X = iris.data # Características
y = iris.target # Etiquetas

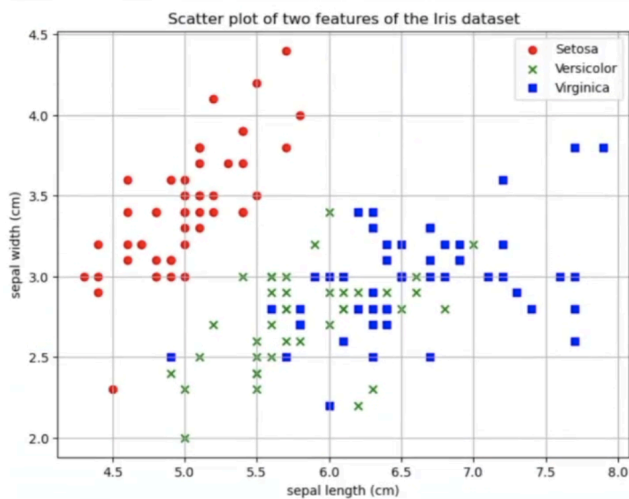
# Seleccionar la primera (índice 0) y segunda (índice 1)
característica para el gráfico
feature1_index = 0
feature2_index = 1

# Creación del gráfico de dispersión (Scatter Plot)
plt.figure(figsize=(8, 6))
# Se añaden las tres clases de flores (Setosa, Versicolor,
Virginica) al gráfico
plt.scatter(X[y == 0, feature1_index], X[y == 0,
feature2_index], label="Setosa", c="r", marker="o")
plt.scatter(X[y == 1, feature1_index], X[y == 1,
feature2_index], label="Versicolor", c="g", marker="x")
plt.scatter(X[y == 2, feature1_index], X[y == 2,
feature2_index], label="Virginica", c="b", marker="s")
```

```
# Añadir etiquetas y título
plt.xlabel(iris.feature_names[feature1_index]) # 'sepal length
(cm)'
plt.ylabel(iris.feature_names[feature2_index]) # 'sepal width
(cm)'
plt.title("Scatter plot of two features of the Iris dataset")

# Añadir leyenda y mostrar gráfico
plt.legend()
plt.grid(True)
plt.show()
```

La ejecución de este bloque de código resultará en un gráfico que se mostrará inmediatamente en el *notebook*.



Matplotlib es una herramienta extremadamente potente que permite dibujar datos en diferentes formatos, como gráficos de barras, líneas o dispersión, como el que acabamos de generar. La **visualización de la información es primordial** en este mundo para entender lo que está sucediendo con los datos y los modelos.

En resumen, **Conda, Jupyter Notebook, Pandas, NumPy, Matplotlib y Scikit-Learn** conforman el arsenal fundamental para cualquier profesional que trabaje en ciencia de datos y IA, y serán las herramientas clave para nuestro trabajo en ciberseguridad.

Ejercicio Práctico: Análisis de Logs de Cortafuegos con Pandas

Bienvenidos a esta nueva sesión, donde pondremos en práctica los fundamentos del análisis de datos utilizando herramientas de IA, concretamente **Pandas**, aplicándolo a un contexto crucial de ciberseguridad: el análisis de logs de un cortafuegos (*firewall*).

Nuestro objetivo central en este ejercicio es aplicar técnicas básicas de análisis exploratorio para **obtener la mayor información posible** a partir de un *dataset* de logs. Esta fase inicial es la base de cualquier proyecto de *Machine Learning* aplicado.

Enunciado del Ejercicio: Análisis de Logs de Cortafuegos

El conjunto de datos a utilizar contiene logs de actividad de un cortafuegos. Este *dataset* puede descargarse y obtenerse desde la siguiente URL pública:

URL del Dataset de Logs de Cortafuegos:

<https://archive.ics.uci.edu/dataset/542/internet+firewall+data>

Pasos a Seguir:

La ejecución de este ejercicio requiere completar una secuencia lógica de pasos, utilizando las herramientas introducidas en sesiones anteriores:

1. **Cargar los datos** dentro de un **DataFrame de Pandas**.
2. **Interpretar los datos** utilizando comandos de análisis exploratorio.
3. **Visualizar un gráfico** para representar la información relevante.
4. **Interpretar el gráfico** y obtener conclusiones o **resultados** relevantes.

A pesar de ser un ejercicio introductorio, se anima a investigar cualquier información adicional necesaria, aunque los comandos esenciales se proporcionan a continuación.

```
print("First rows of the data set:")
print(data.head())

print("Data set info:")
print(data.info())

print("Descriptive statistics of the dataset:")
print(data.describe())
```

Procedimientos Prácticos: Análisis Exploratorio con Pandas

Para guiar la interpretación de los datos (Paso 2), utilizaremos tres métodos fundamentales de la librería Pandas. Estos comandos proporcionan una visión rápida y crucial de la estructura, calidad y distribución estadística del *dataset*, información muy relevante incluso antes de llegar a un análisis más profundo con *Machine Learning*.

Asumimos que el DataFrame ha sido cargado en una variable llamada `data`.

1. Visualización de las Primeras Filas: `data.head()`

- **Propósito:** Este método se utiliza para **mostrar las primeras filas de un DataFrame**. Es esencial para verificar que los datos se han cargado correctamente y para obtener una vista rápida de los encabezados de las columnas y el formato de las entradas.

- **Sintaxis y Comandos:**

```
print("First rows of the data set:")
print(data.head())
```

- **Detalle:** Por defecto, el método `head()` mostrará las **cinco primeras filas** del *DataFrame*. Sin embargo, puedes especificar el número exacto de filas que deseas visualizar pasándole un argumento entero, por ejemplo, `data.head(10)` mostraría las diez primeras filas.

2. Obtención de Información Estructural: `data.info()`

- **Propósito:** Este método proporciona una **descripción concisa del DataFrame**. Es extremadamente útil para obtener una visión general y rápida de la estructura y los tipos de datos.
- **Sintaxis y Comandos:**

```
print("Data set info:")
print(data.info())
```
- **Detalle:** La salida de `info()` incluirá:
 - El **tipo de datos** de cada columna (ej. *integer*, *float*, *object*).
 - La **cantidad de valores no nulos** por columna, lo cual es vital para identificar rápidamente columnas con datos faltantes (valores nulos).
 - La **cantidad total de entradas** en el *DataFrame*.

3. Generación de Estadísticas Descriptivas: `data.describe()`

- **Propósito:** Este método **genera estadísticas descriptivas** para cada columna **numérica** en el *DataFrame*. Proporciona un resumen rápido de la distribución de los datos numéricos.
- **Sintaxis y Comandos:**

```
print("Descriptive statistics of the dataset:")
print(data.describe())
```
- **Detalle:** Para cada columna numérica, `describe()` calculará automáticamente métricas clave como:
 - **count**: Número de valores no nulos.
 - **mean**: La media aritmética.

- **std**: La desviación estándar.
 - **min**: El valor mínimo.
 - **max**: El valor máximo.
 - **Percentiles** (25%, 50% -mediana-, 75%).
-

Indicación para la Visualización (Paso 3)

Una vez que se haya interpretado la estructura del *dataset* mediante los comandos anteriores, el siguiente paso será la visualización (Paso 3).

Como punto de partida para la representación gráfica, se sugiere enfocar el análisis en dos características específicas presentes en el *dataset*: **"Source Port"** (Puerto Origen) y **"Frequency"** (Frecuencia).

Al visualizar la relación entre estas dos variables utilizando **Matplotlib** o herramientas similares, se facilitará la interpretación de patrones en el tráfico del cortafuegos, lo que llevará a la obtención de resultados concluyentes para la ciberseguridad (Paso 4).

Solución del Ejercicio Práctico: Análisis

Exploratorio de Logs de Cortafuegos

En esta sesión, abordaremos la solución detallada al ejercicio de **Análisis Exploratorio de Datos (EDA)** aplicado a logs de cortafuegos. El objetivo es obtener información relevante sobre la actividad de la red utilizando las librerías **Pandas** y **Matplotlib/Seaborn**, sentando la base para futuros análisis con *Machine Learning* en ciberseguridad.

1. Carga de Datos y Preparación del Entorno

El primer paso en cualquier análisis es asegurar la correcta carga del *dataset* en un entorno de trabajo funcional. Para ello, utilizamos un **Jupyter Notebook** con las librerías fundamentales ya instaladas.

A. Importación de Librerías

Inicialmente, importamos Pandas y Matplotlib, utilizando sus alias convencionales (**pd** y **plt**), lo cual es una práctica estándar en la ciencia de datos para simplificar la sintaxis.

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

Propósito: `import pandas as pd` carga la librería para manipular DataFrames.
`import matplotlib.pyplot as plt` carga el módulo de gráficos para visualización.

B. Carga del Dataset en un DataFrame

A continuación, especificamos la ruta del fichero de logs (`log2.csv`) y lo cargamos en un **DataFrame** de Pandas, denominado `data`.

```
file_path = 'log2.csv'

data = pd.read_csv(file_path)
```

Sintaxis Detallada: `pd.read_csv(file_path)` es la función de Pandas que lee el fichero de texto separado por comas y lo transforma en un DataFrame, la estructura principal que permite la manipulación de datos.

2. Inspección Preliminar del Dataset (Toma de Contacto)

Una vez cargado, es fundamental realizar una **toma de contacto** con el *dataset* para comprender su estructura, el tipo de datos que contiene y si presenta anomalías.

A. Visualización de Filas: `data.head()`

Propósito: Mostrar las primeras filas del DataFrame. Esto es crítico para ver los nombres de las columnas (*headers*), el formato de los datos y la información contenida en cada campo.

Comando y Resultado:

```
print(data.head())
```

Al ejecutar `data.head()`, sin argumentos, se muestran las **cinco primeras filas**. La imagen muestra las columnas de **Puertos** (`Source Port`, `Destination Port`, etc.), **Métricas de Tráfico** (`Bytes`, `Packets`, `Elapsed Time (sec)`), y la **Acción** del cortafuegos (`Action`).

```
import pandas as pd
import matplotlib.pyplot as plt

file_path = 'log2.csv'
data = pd.read_csv(file_path)

print(data.head())
```

	Source Port	Destination Port	NAT Source Port	NAT Destination Port	\
0	57222	53	54587	53	
1	56258	3389	56258	3389	
2	6881	50321	43265	50321	
3	50553	3389	50553	3389	
4	50002	443	45848	443	

	Action	Bytes	Bytes Sent	Bytes Received	Packets	Elapsed Time (sec)	\
0	allow	177	94	83	2	30	
1	allow	4768	1600	3168	19	17	
2	allow	238	118	120	2	1199	
3	allow	3327	1438	1889	15	17	
4	allow	25358	6778	18580	31	16	

	pkts_sent	pkts_received
0	1	1
1	10	9
2	1	1
3	8	7
4	13	18

B. Obtención de Información Estructural: `data.info()`

Propósito: Obtener una descripción concisa orientada al formato, incluyendo tipos de datos y valores nulos.

Comando y Resultado:

```
print(data.info())
```

```
print(data.info())

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 65532 entries, 0 to 65531
Data columns (total 12 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   Source Port           65532 non-null  int64
 1   Destination Port      65532 non-null  int64
 2   NAT Source Port       65532 non-null  int64
 3   NAT Destination Port  65532 non-null  int64
 4   Action                65532 non-null  object
 5   Bytes                 65532 non-null  int64
 6   Bytes Sent            65532 non-null  int64
 7   Bytes Received        65532 non-null  int64
 8   Packets               65532 non-null  int64
 9   Elapsed Time (sec)    65532 non-null  int64
10   pkts_sent             65532 non-null  int64
11   pkts_received         65532 non-null  int64
dtypes: int64(11), object(1)
memory usage: 6.0+ MB
None
```

Análisis Detallado de `data.info()`:

- **RangeIndex: 65532 entries:** Indica que el *dataset* contiene un total de 65.532 registros.

- **Non-Null Count:** En este caso, todas las columnas muestran **65532 non-null**, lo cual es un hallazgo excelente, ya que indica que **no hay valores nulos** en el *dataset*.
- **Dtype:** Muestra los tipos de datos. Por ejemplo, la columna **Action** es **object**. El tipo **object** se refiere generalmente a **cadenas de texto (strings)** o a columnas con datos mixtos, mientras que la mayoría de las demás son **int64** (enteros de 64 bits).

C. Análisis Estadístico Descriptivo: **data.describe()**

Propósito: Generar estadísticas descriptivas para las **columnas numéricas**. Esto proporciona una visión rápida de la distribución de los valores.

Comando y Resultado:

```
print(data.describe())
```

```
print(data.describe())
```

	Source Port	Destination Port	NAT Source Port	NAT Destination Port	\
count	65532.000000	65532.000000	65532.000000	65532.000000	
mean	49391.969343	10577.385812	19282.972761	2671.049930	
std	15255.712537	18466.027039	21970.689669	9739.162278	
min	0.000000	0.000000	0.000000	0.000000	
25%	49183.000000	80.000000	0.000000	0.000000	
50%	53776.500000	445.000000	8820.500000	53.000000	
75%	58638.000000	15000.000000	38366.250000	443.000000	
max	65534.000000	65535.000000	65535.000000	65535.000000	

	Bytes	Bytes Sent	Bytes Received	Packets	\
count	6.553200e+04	6.553200e+04	6.553200e+04	6.553200e+04	
mean	9.712395e+04	2.238580e+04	7.473815e+04	1.028660e+02	
std	5.618439e+06	3.828139e+06	2.463208e+06	5.133002e+03	
min	6.000000e+01	6.000000e+01	0.000000e+00	1.000000e+00	
25%	6.600000e+01	6.600000e+01	0.000000e+00	1.000000e+00	
50%	1.680000e+02	9.000000e+01	7.900000e+01	2.000000e+00	
75%	7.522500e+02	2.100000e+02	4.490000e+02	6.000000e+00	
max	1.269359e+09	9.484772e+08	3.208818e+08	1.036116e+06	

	Elapsed Time (sec)	pkts_sent	pkts_received
count	65532.000000	65532.000000	65532.000000
mean	65.833577	41.399530	61.466505
std	302.461762	3218.871288	2223.332271
min	0.000000	1.000000	0.000000
25%	0.000000	1.000000	0.000000
50%	15.000000	1.000000	1.000000
75%	30.000000	3.000000	2.000000
max	10824.000000	747520.000000	327208.000000

Análisis Detallado de `data.describe()`:

- **count**: Confirma que hay 65.532 observaciones no nulas en todas las columnas numéricas.
- **mean**: Muestra el valor promedio. En métricas como `Bytes`, `Packets` o `pkts_sent`, el valor medio (ej. **mean** de `Packets` es ≈ 102.86) sí tiene sentido. Sin embargo, para los campos de **puertos** (`Source Port`, `Destination Port`), el valor medio carece de significado estadístico, ya que los puertos deben ser tratados como campos categóricos o alfanuméricos, no como valores sumables.
- **std**: La desviación estándar.
- **min** / **Cuartiles** / **max**: Muestran el rango de valores, los percentiles (25%, 50% -mediana-, 75%) y los valores extremos.

La interpretación de estas estadísticas es fundamental: por ejemplo, la alta desviación estándar en `Elapsed Time (sec)` (302.4 segundos) en comparación con su media (≈ 65.8 segundos) sugiere que la mayoría de las conexiones son muy cortas, pero existen algunas que duran mucho más tiempo (valores atípicos o *outliers*).

3. Visualización de Datos para la Detección de Anomalías

La visualización es el paso más importante antes de aplicar *Machine Learning*, ya que permite detectar patrones y anomalías a simple vista. Nos centraremos en la distribución de métricas clave de tráfico.

A. Gráficos de Caja (*Boxplots*) para Detección de *Outliers*

Un **Boxplot** (*gráfico de caja y bigotes*) es ideal para visualizar la mediana, los cuartiles y, crucialmente, los **valores atípicos** (*outliers*), que se representan como puntos individuales fuera de los "bigotes" o whiskers.

Selección de Columnas de Interés: Se seleccionan las columnas numéricas relacionadas con el volumen y duración del tráfico.

```
column_of_interest = ['Bytes', 'Bytes Sent', 'Bytes Received',  
'Packets', 'pkts_sent', 'pkts_received', 'Elapsed Time (sec)']
```

Generación de Subgráficos de Boxplot: Se utiliza un bucle para generar un *boxplot* para cada columna.

```
plt.figure(figsize=(12,8)) # Establece el tamaño de la figura  
(12x8 pulgadas)  
  
for i, col in enumerate(column_of_interest):  
    plt.subplot(2, 4, i+1) # Crea un subgráfico en la posición  
(2 filas, 4 columnas, índice i+1)  
    plt.boxplot(data[col]) # Traza el boxplot de la columna  
actual  
    plt.title(col)          # Establece el título del  
subgráfico  
  
plt.tight_layout() # Ajusta automáticamente los parámetros del  
gráfico para un diseño apretado  
plt.show()
```

```
column_of_interest = ['Bytes', 'Bytes Sent', 'Bytes Received', 'Packets', 'pkts_sent', 'pkts_received', 'Elapsed Time (sec)']

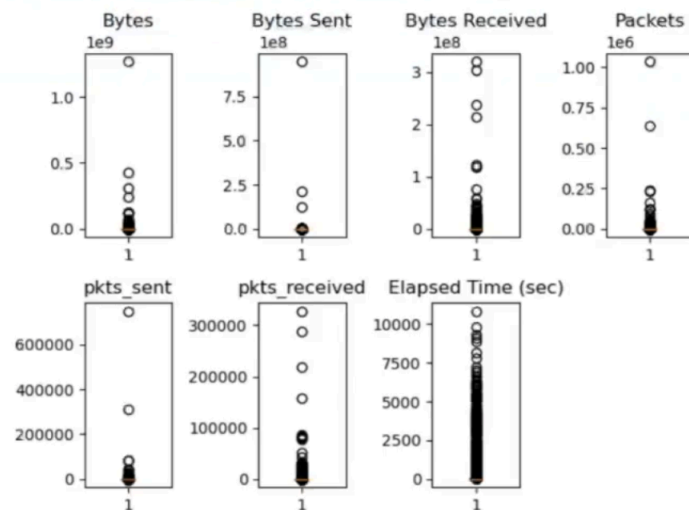
plt.figure(figsize=(12,8))

<Figure size 1200x800 with 0 Axes>
<Figure size 1200x800 with 0 Axes>

for i, col in enumerate(column_of_interest):
    plt.subplot(2, 4, i+1)
    plt.boxplot(data[col])
    plt.title(col)

plt.tight_layout()
plt.show

<function matplotlib.pyplot.show(close=None, block=None)>
```



Conclusión del Boxplot: Los gráficos para **Bytes**, **Bytes Sent**, **Bytes Received**, y **Packets** muestran una distribución **muy sesgada** (la caja es muy pequeña y está cerca de cero). Esto indica que la inmensa mayoría de las transacciones son muy pequeñas. Los **círculos individuales** que se extienden a lo largo de las gráficas son **valores atípicos** (*outliers*), que representan **transferencias de datos significativamente altas**. Estos picos son de gran interés para un analista de ciberseguridad, ya que podrían ser tráfico legítimo inusual o evidencia de un ataque de transferencia masiva de datos (ej. exfiltración).

B. Histograma de Distribución de Puertos Origen (*Source Port*)

Aunque el valor promedio de los puertos no es útil, su **frecuencia de uso** es clave para identificar comportamientos comunes o inusuales.

Comandos para Generar el Histograma:

```
plt.figure(figsize=(10,6))
```

```
plt.hist(data['Source Port'], bins=30, color='blue') # bins=30
```

```
define 30 intervalos (barras)
```

```
plt.xlabel('Source Port')
```

```
plt.ylabel('Freq')
```

```
plt.title('Source Port Distribution')
```

```
plt.show()
```

```
plt.figure(figsize=(10,6))
```

```
<Figure size 1000x600 with 0 Axes>
```

```
<Figure size 1000x600 with 0 Axes>
```

```
plt.hist(data['Source Port'], bins=30, color='blue')
```

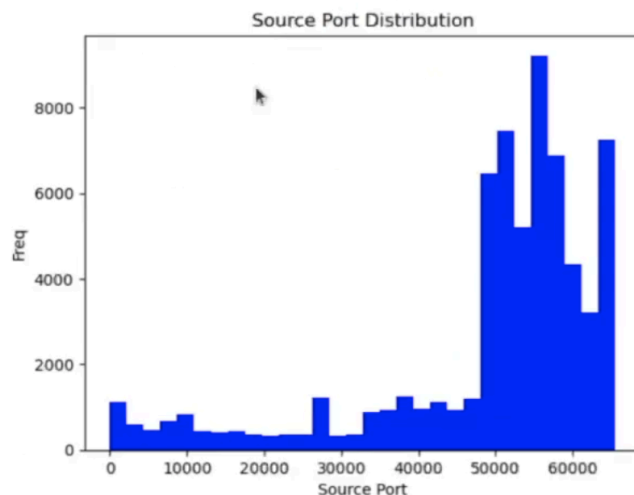
```
plt.xlabel('Source Port')
```

```
plt.ylabel('Freq')
```

```
plt.title('Source Port Distribution')
```

```
plt.show
```

```
<function matplotlib.pyplot.show(close=None, block=None)>
```



Conclusión del Histograma: La altura de las barras indica la **frecuencia de uso** de un rango de puertos de origen. Se observa una concentración muy alta de actividad en el rango de **puertos efímeros/dinámicos** (generalmente entre el 49152 y el 65535, que corresponde a los picos entre 50000 y 60000). Esto es un comportamiento normal para el tráfico de salida iniciado por clientes, pero el análisis de los picos puede revelar puertos específicos utilizados con inusual frecuencia.

C. Mejora de la Visualización: Histogramas Segmentados con Seaborn

Para obtener una visión más clara y **comparativa** de las métricas de tráfico, y para confirmar visualmente las conclusiones iniciales extraídas del *boxplot* respecto a la asimetría del tráfico, se puede **segmentar la información** mediante dos histogramas independientes utilizando la librería **Seaborn**. Seaborn es una biblioteca basada en Matplotlib que proporciona una interfaz de alto nivel para crear gráficos estadísticos más atractivos e informativos.

Preparación y Segmentación: Utilizaremos la función `plt.subplot` de Matplotlib, combinada con `sns.histplot` de Seaborn, para dividir la figura en dos y comparar directamente los volúmenes de **Bytes Sent** (Bytes Enviados) y **Bytes Received** (Bytes Recibidos).

Generación de Histogramas Segmentados: Se generan dos subgráficos, utilizando una **escala logarítmica** en el eje Y (**Count** o Frecuencia) para manejar el vasto rango de valores y hacer visibles las ocurrencias raras o picos de actividad.

```
import seaborn as sns

# Se establece el tamaño de la figura antes de los subplots
plt.figure(figsize=(10, 6))

# Primer Histograma: 'Bytes Sent'
plt.subplot(1, 2, 1) # Crea el primer subgráfico (1 fila, 2
columnas, posición 1)
# sns.histplot: Función de histograma de Seaborn. bins=50:
define la granularidad.
sns.histplot(data['Bytes Sent'], bins=50, kde=False,
color='blue')
plt.title('Histogram of Bytes Sent')
```

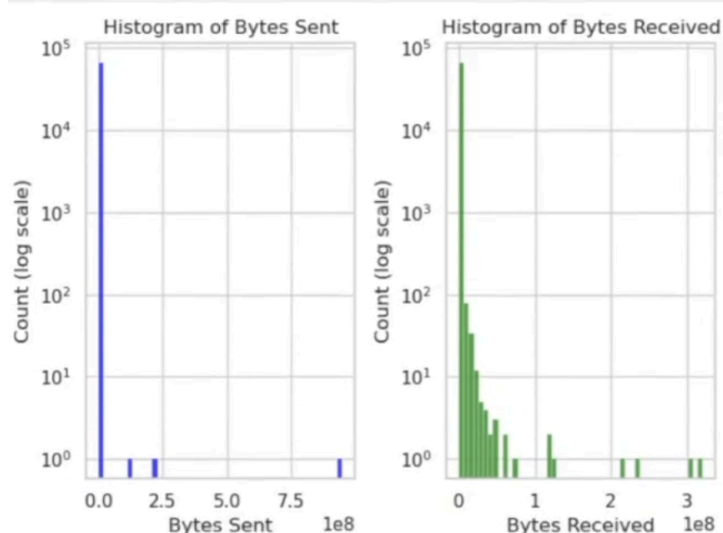
```

# plt.yscale('log'): Aplica escala logarítmica al eje Y
(frecuencia)
plt.yscale('log')
plt.xlabel('Bytes Sent')
plt.ylabel('Count (log scale)')

# Segundo Histograma: 'Bytes Received'
plt.subplot(1, 2, 2) # Crea el segundo subgráfico (1 fila, 2
columnas, posición 2)
sns.histplot(data['Bytes Received'], bins=50, kde=False,
color='green')
plt.title('Histogram of Bytes Received')
plt.yscale('log')
plt.xlabel('Bytes Received')
plt.ylabel('Count (log scale)')

# Ajusta automáticamente los parámetros del subgráfico para
que no se superpongan
plt.tight_layout()
plt.show()

```



- **Conclusión de la Segmentación:** Esta visualización confirma claramente la deducción inicial: la distribución del tráfico es **asimétrica**. Se observa una frecuencia notablemente mayor de **Bytes Recibidos** (gráfico de la derecha) en comparación con los **Bytes Enviados** (gráfico de la izquierda). Esto sugiere que el flujo de datos predominante en el cortafuegos es de entrada o de respuesta a peticiones, lo cual es relevante para identificar posibles patrones de red anómalos o comportamientos típicos de descarga/navegación intensa.

D. Diagrama de Dispersión (*Scatter Plot*) para Correlación de Tráfico

Para visualizar la relación entre los datos enviados y recibidos, utilizamos un **diagrama de dispersión** con **escala logarítmica** para manejar el vasto rango de valores (varios órdenes de magnitud) visto en los Boxplots.

Comandos para Generar el Diagrama de Dispersión:

```
plt.figure(figsize=(10, 6))

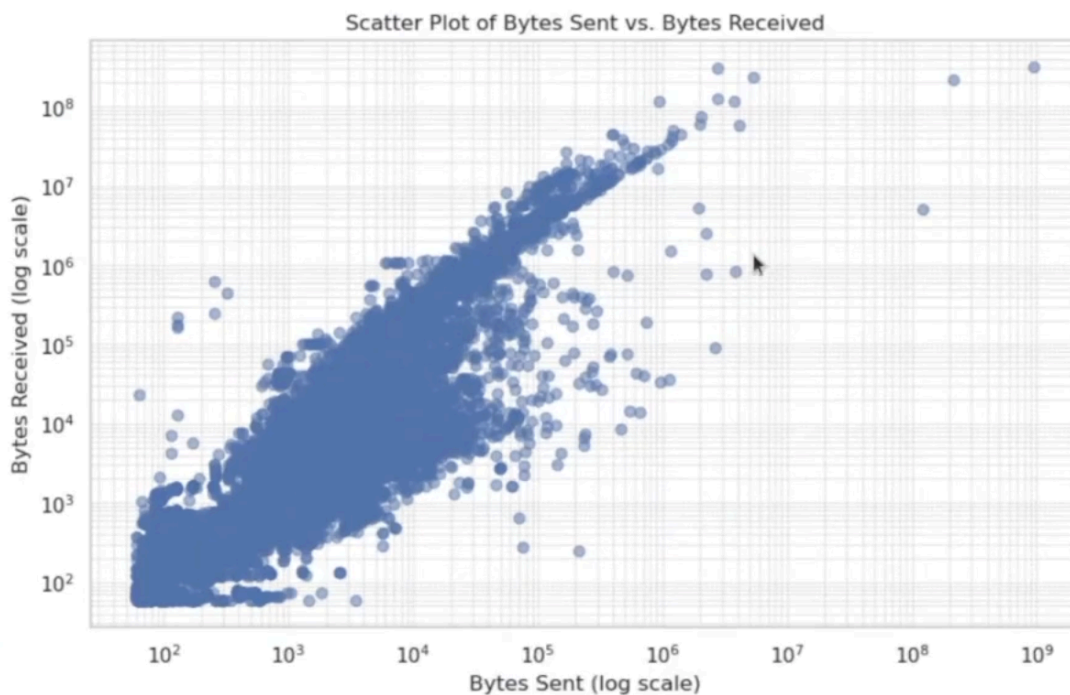
plt.scatter(data['Bytes Sent'], data['Bytes Received'],
            alpha=0.5)

plt.xscale('log') # Eje x en escala logarítmica
plt.yscale('log') # Eje y en escala logarítmica

plt.title('Scatter Plot of Bytes Sent vs. Bytes Received')
plt.xlabel('Bytes Sent (log scale)')
plt.ylabel('Bytes Received (log scale)')

plt.grid(True, which="both", ls="--", linewidth=0.5)
```

```
plt.show()
```



Conclusión del Diagrama de Dispersión:

-Correlación Positiva: La agrupación de puntos a lo largo de la diagonal central demuestra una **fuerte correlación positiva** entre los bytes enviados y los recibidos (la mayoría de las comunicaciones son bidireccionales y equilibradas).

-Identificación de Anomalías: Los puntos que se desvían significativamente de esta diagonal central, especialmente aquellos donde hay muchos **Bytes Sent** y pocos **Bytes Received** (o viceversa), indican **comportamientos atípicos**. Estos podrían ser el resultado de **ataques de denegación de servicio (DoS)** o **exfiltración de datos**, donde la proporción de tráfico se rompe. Estos son los puntos que un modelo de *Machine Learning* intentaría clasificar como anómalos.

4. Conclusiones Fundamentales

El análisis exploratorio y la visualización son pasos **imprescindibles** antes de aplicar cualquier algoritmo de IA:

- **Inspección Detallada:** Comandos como `head()`, `info()`, y `describe()` son cruciales para entender la estructura, la integridad (ausencia de nulos) y la distribución estadística del *dataset*.
- **Detección de Anomalías:** El uso de gráficos adecuados, como el **Boxplot** (para *outliers*) y el **Diagrama de Dispersión Logarítmica** (para correlaciones), permitió identificar que la mayoría del tráfico es pequeño y equilibrado, pero existen transferencias masivas y desequilibradas que requieren un análisis profundo, ya que pueden ser indicadores de intrusiones o actividad maliciosa.
- **Visualización como Base de la IA:** La visualización de datos con librerías como Matplotlib y Seaborn no solo ayuda a **comprender** los datos, sino que también **guía la elección** del algoritmo de *Machine Learning* más apropiado para la posterior clasificación de este tráfico.